

Homework 3: Pumping Lemma and Context Free Grammars

Devin Fromond

Due: 2/3/2025

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments. Three bonus points if you use $\text{\LaTeX}$, and our template. You may collaborate with other students, but any written work should be your own.

1. Let D_1, D_2 be two DFAs of n, m states respectively, both with the same alphabet. Find a function $f(n, m)$ and prove the following. If $L(D_1) \neq L(D_2)$ then the shortest string they disagree on has length $\leq f(n, m)$. Discuss the consequences of the contrapositive of this statement.

Answer: We define w as the shortest string such that D_1 and D_2 disagree. Similarly, we define the sequence of states visited by each DFA as they process w . For D_1 , we have the following: $q_0^1, q_1^1, \dots, q_k^1$. Similarly, for D_2 , we have the following: $q_0^2, q_1^2, \dots, q_k^2$. We are able to turn these states into pairs in the form of $(q_0^1, q_0^2), \dots, (q_k^1, q_k^2)$. Trivially, we find there are $|w| + 1$ pairs.

If any pair repeats, we have the following: $(q_i^1, q_i^2) = (q_j^1, q_j^2)$ for some integer where $i < j$. Relatively trivially, we find this occurrence will signify a loop within the DFA. Therefore, we find the portion of w between i and j can be removed, thereby resulting in a shorter distinct string. We find this statement contradicts the minimal nature of w . Therefore, we find all $k + 1$ pairs must be distinct, where $k = |w|$.

Since there exists at most $n \cdot m$ distinct pairs, we find $k + 1 \leq n \cdot m$ which equates to $k \leq n \cdot m - 1$. Therefore, we find if $L(D_1) \neq L(D_2)$, then there exists a string of length $\leq f(n, m) = n \cdot m - 1$.

We find the contrapositive of the statement has the consequence of if D_1 and D_2 agree on strings of length $\leq f(n, m) = n \cdot m - 1$, then they are equivalent. Therefore, to determine equivalency, we must only check strings of length $\leq n \cdot m - 1$.

2. Prove that the following language is not regular using the pumping lemma:

$$L = \{1^n \mid n \text{ is prime}\}.$$

Answer: We will assume L is regular. By the Pumping Lemma, we find there must exist a pumping length p such that every string $s \in L$ with $|s| \geq p$ can be represented as $s = xyz$ where $|xy| \leq p$, $|y| \geq 1$, and $xy^i \in L$ for $i \geq 0$.

We define a prime number q such that $q \geq p$ where $s = 1^q \in L$. Therefore, since $|xy| \leq p \leq q$, we find the segment y contains only 1 's, where $|y| = k \geq 1$. Therefore, we have the following: $xy^iz = 1^{q+(i-1)k}$. If we set $i = q + 1$, we find $xy^{q+1}z = 1^{q(1+k)}$. As previously defined, we have $k \geq 1$. However, since $q(1+k)$ is a composite, we find the string $1^{q(1+k)}$ is not in L . We therefore have contradicted the pumping lemma.

Therefore, we find our assumption that L is regular is false and so, L is not regular.

3. (a) Give CFGs for each of the following languages: $\{\lambda\}$, $\emptyset$, $\{a\}$, $\{b\}$.

Answer:

- For $\{\lambda\}$: A possible CFG is the language containing only the empty string such as $S \rightarrow \varepsilon$.
- For $\emptyset$: A possible CFG is a production that never produces a terminal such as $S \rightarrow S$.
- For $\{a\}$: A possible CFG is the language containing only the string a such as $S \rightarrow a$.
- For $\{b\}$: A possible CFG is the language containing only the string b such as $S \rightarrow b$.

- (b) Prove that the class of context-free languages is closed under union, concatenation, and Kleene star.

Answer: We define $G_1 = (V_1, \Sigma, R_1, S_1)$ as a CFG for L_1 and $G_2 = (V_2, \Sigma, R_2, S_2)$ as a CFG for L_2 . We will assume V_1 and V_2 are disjoint. Therefore, we construct a new CFG G such that $G = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1 | S_2\}, S)$. The symbol S derives either a string from L_1 or L_2 . Therefore, we find G generates $L_1 \cup L_2$. We find closure under union.

Using the same definitions of G_1 and G_2 from above, we define the CFG G such that $G = (V_1 \cup V_2 \cup \{S\}, \Sigma, R_1 \cup R_2 \cup \{S \rightarrow S_1 S_2\}, S)$. Therefore, we find the symbol S produces a string generated by S_1 (which is a string in L_1 followed by a string generated by S_2 (which is a string in L_2). Therefore, we find G generates $L_1 L_2$. We find closure under concatenation.

we define a new G such that $G = (V, \Sigma, R, S_1)$ is a CFG for L . We wish to construct a new CFG for L^* by including a new S and adding the productions of $S \rightarrow S_1 S$ and $S \rightarrow \varepsilon$. Therefore, we find the new CFG to be defined as $G' = (V \cup \{S\}, \Sigma, R \cup \{S \rightarrow S_1 S, S \rightarrow \varepsilon\}, S)$. We find $S \rightarrow S_1 S$ allows generation of one string from L followed by further strings from L^* with the production $S \rightarrow \varepsilon$ being the base-case that allows for termination. Therefore, we find G' generates L^* . We find closure under Kleene star.

- (c) Why does this imply every regular language is context-free? **Answer:** Every regular language can be described by a regular expression, which are built from basic languages as described in 3a and using the operations from 3b. Therefore,

we find context free languages are closed under the aforementioned operations and we find each basic language described in 3a is context free. Therefore, by induction, we find every regular language is context free.

4. Give CFGs for the following:

- (a) $\{a^n b^m \mid n \neq m\}$

Answer: We begin by splitting L into L_1 and L_2 defined as $L_1 = \{a^n b^m \mid n > m\}$ and $L_2 = \{a^n b^m \mid n < m\}$. We define the productions as follows:

$$S \rightarrow S_1 \mid S_2$$

For L_1 , we have the following:

$$S_1 \rightarrow aS_1b \mid T$$

$$T \rightarrow aT \mid a$$

For L_2 , we have the following:

$$S_2 \rightarrow aS_2b \mid U$$

$$U \rightarrow bU \mid b$$

We observe that for L_1 , the production $S_1 \rightarrow aS_1b \mid T$ adds one a or one b , ensuring they are balanced. The other production maintains $n > m$ by only adding a 's. We trivially observe a similar production applies to L_2 . Therefore, we find the above productions generate all strings in the form $a^n b^m$ with $n \neq m$.

- (b) $\{a^i b^j \mid x, y \in \mathbb{N}, x + y = i \text{ and } 4x + 3y = j\}$.

Answer: We find if we create variables n, r such that $n = x + y$ and $r = x$, then we have the following expression $j = 4x + 3y = 4r + 3(n - r) = 3n + r$ for $0 \leq r \leq n$. Therefore, we find the strings must be in the form of $a^n b^{3n+r}$ for $0 \leq r \leq n$.

Therefore, we arrive at the CFG of $S \rightarrow \varepsilon \mid aSbbb \mid aSbbbb$. We find n steps produces n a 's. Similarly, if r steps use the $bbbb$ production, then we find the total number of b 's to be $(n - r) \cdot 3 + r \cdot 4 = 3n + r$, as previously described, with any r between 0 and n is valid.

- (c) The complement of $\{a^n b^n \mid n \in \mathbb{N}\}$ ¹

Answer: By definition, a string $w \in \{a, b\}^*$ is not in $\{a^n b^n\}$ iff it is not in the form $a^* b^*$ or it is not in the form $a^n b^m$, where $n \neq m$. We observe "all strings not in $a^* b^*$ " and the grammar as defined in 4a are context-free. Therefore, we define S as the following:

$$S \rightarrow S_1 \mid S_2$$

¹CFLs are not closed under intersection or complement in general, this one is just special.

where S_1 generates strings not in a^*b^* , which has the productions of:

$$S_1 \rightarrow AbBaC$$

$$A \rightarrow aA \mid bA \mid \varepsilon$$

$$B \rightarrow aB \mid bB \mid \varepsilon$$

$$C \rightarrow aC \mid bC \mid \varepsilon$$

such that b comes before a . Similarly, we define the productions for S_2 defined as:

$$S_2 \rightarrow S_{2,1} \mid S_{2,2}$$

$$S_{2,1} \rightarrow aS_{2,1}b \mid T, \quad T \rightarrow aT \mid a$$

$$S_{2,2} \rightarrow aS_{2,2}b \mid U, \quad U \rightarrow bU \mid b$$

such that each string not in the form $a^n b^n$ will be generated by S_1 or S_2 .

- (d) $\{w \in \Sigma^* \mid \#a(w) + \#b(w) = \#c(w) + \#d(w)\}$ with $\Sigma = \{a, b, c, d\}$

Answer: We wish to match an occurrence of a or b with an occurrence of c or d. Therefore, with tedious work, we find the following:

$$S \rightarrow aSc \mid aSd \mid bSc \mid bSd \mid cSa \mid cSb \mid dSa \mid dSb \mid SS \mid \varepsilon$$

The above production matches the a or b on the right to its corresponding c or d on the right. Similarly, we find the SS production ensures two balanced pieces may be concatenated. Therefore, we find that every string generated has $\#a + \#b = \#c + \#d$.